Uniwersytet Bielsko-Bialski

Wydział Budowy Maszyn i Informatyki

Technologie Blockchain
(ćwiczenia laboratoryjne nr 2)

Opracowanie: mgr inż. Marcel Gańczarczyk

Temat: Transakcje i wydobywanie bloków w prototypowym łańcuchu bloków

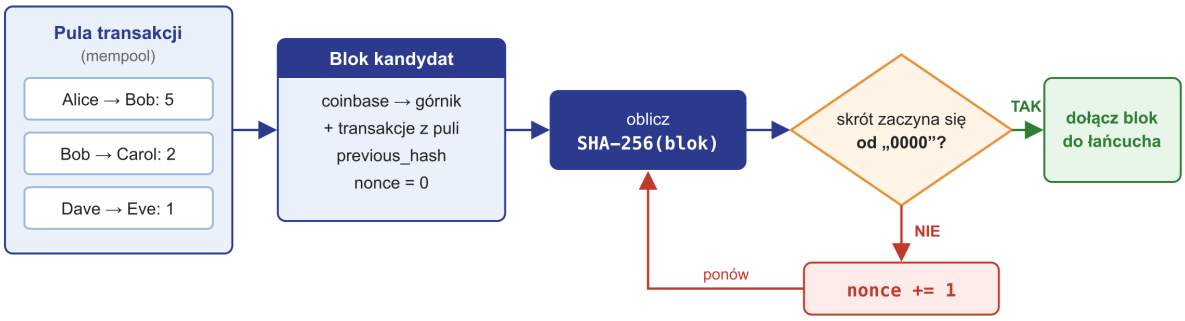
Cel ćwiczenia: Rozszerzenie prototypu blockchaina o pulę transakcji oczekujących i mechanizm wydobywania bloków Proof of Work oraz zbadanie wpływu trudności na koszt obliczeniowy i bezpieczeństwo łańcucha.

Czas realizacji: 2 godz.

1. Wprowadzenie

Transakcja to zapis przekazania wartości od nadawcy (`sender`) do odbiorcy (`recipient`). Nowe transakcje nie trafiają od razu do łańcucha – najpierw oczekują w puli transakcji niepotwierdzonych (w sieci Bitcoin nazywanej *mempool*). Dopiero **górnik** (ang. *miner*) grupuje je w nowy blok i dołącza ten blok do łańcucha.

Wydobywanie (ang. *mining*) polega na znalezieniu takiej wartości `nonce`, aby skrót bloku spełniał warunek trudności – w naszym prototypie skrót musi zaczynać się od określonej liczby zer. Ponieważ funkcja skrótu jest nieprzewidywalna, jedyną metodą jest metoda prób i błędów: zwiększanie `nonce` i ponowne liczenie skrótu. Znalezienie rozwiązania wymaga więc wielu obliczeń, a jego sprawdzenie – tylko jednego. Ta asymetria jest istotą mechanizmu **Proof of Work** (PoW).



Rys. 1. Proces wydobywania bloku w mechanizmie Proof of Work

Każde dodatkowe zero szesnastkowe na początku skrótu zwiększa średnią liczbę prób 16-krotnie: dla trudności *d* potrzeba średnio 16^d prób. W sieci Bitcoin warunek ma postać „skrót nagłówka bloku ≤ wartość docelowa (*target*)”, a trudność jest automatycznie dostosowywana co 2016 bloków tak, aby nowy blok powstawał średnio co 10 minut.

PoW ma bezpośrednie znaczenie dla bezpieczeństwa: zmiana dowolnej transakcji w bloku zmienia jego skrót, więc atakujący musi ponownie wydobyć ten blok **oraz wszystkie bloki po nim**. Koszt ataku rośnie zatem wraz z liczbą potwierdzeń transakcji. Blok zawierający transakcję daje pierwsze potwierdzenie, a każdy blok dobudowany nad nim – kolejne.

W rzeczywistej sieci atakujący nie przelicza łańcucha w spokoju – w tym czasie uczciwi górnicy dobudowują kolejne bloki, a węzły przyjmują łańcuch, w który włożono najwięcej pracy (w ćwiczeniu nr 4 – najdłuższy poprawny łańcuch). Atakujący musi więc dogonić i wyprzedzić resztę sieci. Jeżeli dysponuje mniej niż połową jej mocy obliczeniowej, prawdopodobieństwo sukcesu maleje wykładniczo wraz z liczbą potwierdzeń (S. Nakamoto, 2008, rozdz. 11) – np. przy 10% mocy i 6 potwierdzeniach spada poniżej 0,1%.

Górnik otrzymuje nagrodę za wydobycie bloku w postaci specjalnej transakcji **coinbase**, która jest pierwszą transakcją w bloku i nie ma nadawcy – w prototypie oznaczymy go wartością `"0"`.

Przykład 1: Pula transakcji

Rozszerzamy konstruktor klasy z ćwiczenia nr 1 o pulę transakcji oczekujących i parametr trudności oraz dodajemy metodę `new_transaction()`. Metody `create_genesis_block()`, `hash()` i `last_block` pozostają bez zmian, natomiast metodę `add_block()` zastąpi metoda `mine()`:

```
class Blockchain:
    def __init__(self, difficulty=4):
        self.chain = []
        self.current_transactions = []   # pula transakcji oczekujących
        self.difficulty = difficulty     # liczba wiodących zer skrótu
        self.create_genesis_block()

    def new_transaction(self, sender, recipient, amount):
        self.current_transactions.append({
            "sender": sender,
            "recipient": recipient,
            "amount": amount,
        })
        # numer bloku, do którego trafi transakcja
        return self.last_block["index"] + 1
```

Przykład 2: Proof of Work i wydobycie bloku

```
    def proof_of_work(self, block):
        block["nonce"] = 0
        computed_hash = self.hash(block)
        while not computed_hash.startswith("0" * self.difficulty):
            block["nonce"] += 1
            computed_hash = self.hash(block)
        return computed_hash

    def mine(self, miner_address):
        # Transakcja coinbase (nagroda dla górnika) jako pierwsza w bloku
        reward = {"sender": "0", "recipient": miner_address, "amount": 1}
        block = {
            "index": len(self.chain) + 1,
            "timestamp": time(),
            "transactions": [reward] + self.current_transactions,
            "nonce": 0,
            "previous_hash": self.hash(self.last_block),
        }
        self.proof_of_work(block)
        self.current_transactions = []
        self.chain.append(block)
        return block
```

Metoda `proof_of_work()` zwiększa `nonce` tak długo, aż skrót całego bloku zacznie się od `difficulty` zer. Metoda `mine()` buduje nowy blok z transakcji oczekujących (poprzedzonych transakcją coinbase), wydobywa go, dołącza do łańcucha i czyści pulę.

Przykład 3: Weryfikacja łańcucha z uwzględnieniem PoW

Poprawny łańcuch musi teraz spełniać dwa warunki: powiązania `previous_hash` muszą być zgodne, a skrót każdego bloku (poza blokiem genezy) musi spełniać warunek trudności. Metoda otrzymuje łańcuch jako parametr – w ćwiczeniu nr 4 wykorzystamy ją do sprawdzania łańcuchów otrzymanych od innych węzłów:

```
    def valid_chain(self, chain):
        if chain[0] != self.chain[0]:
            return False  # inny blok genezy = inna sieć
        for i in range(1, len(chain)):
            previous, block = chain[i - 1], chain[i]
            if block["previous_hash"] != self.hash(previous):
                return False
            if not self.hash(block).startswith("0" * self.difficulty):
                return False
        return True
```

Przykład 4: Uruchomienie

```
if __name__ == "__main__":
    bc = Blockchain(difficulty=4)
    bc.new_transaction("Alice", "Bob", 5)
    bc.new_transaction("Bob", "Carol", 2)
    start = time()
    block = bc.mine(miner_address="miner-1")
    print(f"Blok {block['index']} wydobyty w {time() - start:.2f} s, "
          f"nonce={block['nonce']}")
    print("hash:", bc.hash(block))
    print("Łańcuch poprawny:", bc.valid_chain(bc.chain))
```

Wynik:

```
Blok 2 wydobyty w 0.19 s, nonce=42213
hash: 0000d463bc201e011c81b9e79e518b6cd69571680fd5f09b649ab5e9b5d80f7d
Łańcuch poprawny: True
```

Wartość `nonce` i czas wydobycia będą u każdego inne – zależą od zawartości bloku (m.in. znacznika czasu) i wydajności komputera.

2. Zadania do zrealizowania

2.1. Implementacja transakcji i wydobywania

- Rozbuduj klasę `Blockchain` z ćwiczenia nr 1 zgodnie z Przykładami 1–3.
- Dodaj kilka transakcji, wydobądź co najmniej 3 bloki i wyświetl cały łańcuch. Sprawdź, że skrót każdego bloku zaczyna się od wymaganej liczby zer.

2.2. Badanie wpływu trudności

- Dla trudności od 1 do 6 wydobądź po 5 bloków (dla trudności 6 wystarczą 1–2 bloki) i zmierz średni czas wydobycia oraz średnią wartość `nonce`. Uzupełnij poniższą tabelę:

Trudność d	Teoretyczna liczba prób 16^d	Średnia wartość nonce	Średni czas [s]
1	16		
2	256		
3	4 096		
4	65 536		
5	1 048 576		
6	16 777 216		

- Oblicz wydajność swojego komputera w skrótach na sekundę. Przedstaw wyniki na wykresie (np. z użyciem biblioteki `matplotlib`, oś Y w skali logarytmicznej) i porównaj je z wartościami teoretycznymi.
- Na podstawie aktualnej mocy obliczeniowej sieci Bitcoin (np. https://mempool.space/mining) oszacuj, ile lat zajęłoby Twojemu komputerowi wydobycie jednego bloku Bitcoin.

2.3. Atak na integralność łańcucha

- Wydobądź łańcuch złożony z 5 bloków (trudność 4), a następnie zmień kwotę transakcji w bloku nr 2. Pokaż, że `valid_chain()` zwraca `False`.
- Napisz funkcję symulującą atakującego, który po modyfikacji ponownie wydobywa blok nr 2 i wszystkie kolejne bloki. Zmierz czas ataku dla trudności 4 i 5.
- Wyjaśnij, jak czas ataku zależy od trudności i od liczby bloków dobudowanych nad zmodyfikowanym blokiem. Dlaczego w sieci Bitcoin przyjmuje się, że transakcja z 6 potwierdzeniami jest praktycznie nieodwracalna?

2.4. Walidacja transakcji

- Dodaj w metodzie `new_transaction()` walidację danych: kwota musi być liczbą dodatnią, nadawca i odbiorca muszą być niepustymi ciągami znaków i nie mogą być sobie równi. Niepoprawne transakcje powinny zgłaszać wyjątek `ValueError`.
- Zaimplementuj metodę `get_balance(address)` obliczającą saldo adresu na podstawie wszystkich transakcji w łańcuchu (i w puli oczekujących). Odrzucaj transakcje, na które nadawca nie ma środków. Wyjaśnij, dlaczego ten mechanizm chroni przed podwójnym wydawaniem (*double spending*) tylko w obrębie jednego węzła i co w sieci rozproszonej musi dodatkowo zapewnić, że wszystkie węzły mają tę samą historię transakcji (por. ćwiczenie nr 4).

2.5. Zadanie dla chętnych – drzewo Merkle

- Zaimplementuj obliczanie korzenia drzewa Merkle (*Merkle root*) z listy transakcji i zapisuj go w bloku. Wyjaśnij, w jaki sposób drzewo Merkle umożliwia lekkim klientom (SPV) weryfikację obecności transakcji w bloku bez pobierania całego bloku.

2.6. Sprawozdanie

- Przygotuj sprawozdanie zawierające: kod źródłowy (lub link do repozytorium), zrzuty ekranu potwierdzające wykonanie poszczególnych zadań, wyniki pomiarów oraz odpowiedzi na pytania postawione w zadaniach i pytania kontrolne.

3. Pytania kontrolne

1. Na czym polega asymetria kosztu obliczeniowego w mechanizmie Proof of Work?
2. Ile razy średnio wzrasta liczba prób po zwiększeniu trudności o jedno zero szesnastkowe?
3. Czym jest transakcja coinbase i dlaczego jest umieszczana w bloku jako pierwsza?
4. Dlaczego koszt modyfikacji transakcji rośnie wraz z liczbą potwierdzeń?
5. Na czym polega podwójne wydawanie (*double spending*)?